



Trabajo de Fin de Grado

Grado Superior Doble en Desarrollo de Aplicaciones Web y Multiplataforma

JHTTP

Desarrollo de un servidor HTTP/HTTPS en Java.

Autor: *Sergio de Santiago Redondo*

Tutor: *Daniel Sierra Solís*

Curso: *2025/2026*

Madrid, 15 de junio de 2026

Agradecimientos

A mis padres, por apoyarme y animarme estos años.

Í N D I C E

LISTADO DE ABREVIATURAS, SIGLAS Y ACRÓNIMOS.....	3
INTRODUCCIÓN.....	4
1.1. Descripción.....	4
1.2. Breve descripción de los otros capítulos de la memoria.....	4
JUSTIFICACIÓN DEL PROYECTO. BENEFICIARIOS.....	5
OBJETIVOS.....	6
3.1. Objetivos principales.....	6
3.2. Objetivos secundarios.....	6
RECURSOS.....	7
4.1. Software.....	7
4.2. Hardware.....	7
4.3. Humanos.....	8
DESARROLLO DEL PROYECTO.....	9
5.1. Requisitos.....	9
a. Requisitos funcionales.....	9
b. Requisitos no funcionales.....	11
5.2. Arquitectura.....	13
a. Arranque de la aplicación.....	13
b. Arquitectura HTTP y HTTPS.....	14
c. Aceptación de conexiones y concurrencia.....	14
d. Ciclo de vida de una petición.....	15
e. Sistema de módulos.....	15
f. Modelo de datos HTTP.....	16
5.3. Construcción.....	16
5.4. Despliegue.....	17
CONCLUSIONES.....	18
6.1. Cumplimiento de objetivos.....	18
6.2. Futuras mejoras.....	19
6.3. Conclusión final.....	19
REFERENCIAS.....	20

LISTADO DE ABREVIATURAS, SIGLAS Y ACRÓNIMOS

Término	Significado
HTTP	Hypertext Transfer Protocol (Protocolo de Transferencia de Hipertexto)
JVM	Java Virtual Machine (Máquina Virtual de Java)
CRLF	Carriage Return Line Feed (Avance de línea de retorno de carro)
URI	Uniform Resource Identifier (Identificador de Recursos Uniforme)

CAPÍTULO 1

INTRODUCCIÓN

1.1. Descripción

El presente proyecto tiene como finalidad el desarrollo de un servidor web funcional en Java, construido desde cero sin el uso de librerías especializadas.

1.2. Breve descripción de los otros capítulos de la memoria

A continuación, se realizará una breve descripción de los contenidos abordados en cada uno de los otros capítulos de los que se compone esta memoria:

- **Justificación del proyecto. Beneficiarios:** se exponen las razones que motivan el desarrollo de este proyecto y se identifican los principales beneficiarios del mismo.
- **Objetivos:** se detallan tanto los objetivos principales como los objetivos secundarios para llevar a cabo el desarrollo completo del proyecto.
- **Recursos:** se enumeran y describen los recursos técnicos y humanos utilizados.
- **Desarrollo del proyecto:** se expone el proceso de planificación, diseño e implementación del sistema.
- **Conclusiones:** se evalúan los resultados obtenidos, se reflexiona sobre el proceso y se proponen futuras mejoras.

CAPÍTULO 2

JUSTIFICACIÓN DEL PROYECTO. BENEFICIARIOS

La idea surge a raíz de un proyecto desarrollado durante el curso, en el que se implementó un servidor web de carácter muy básico. Dicho proyecto despertó el interés por retomar la idea y llevarla hasta un nivel de completitud real, pudiendo expandir sus capacidades si se requiriese. Se plantea por tanto como una oportunidad para profundizar y asentar los conocimientos adquiridos a lo largo del grado, abordando de forma práctica conceptos como la concurrencia, la gestión de recursos o la comunicación en red.

Este proyecto no busca competir con soluciones ya consolidadas en el mercado, como *Apache* o *NGINX*, sino comprender e implementar de forma práctica las piezas principales que intervienen en un servidor *HTTP/HTTPS*. Sin embargo, y debido a su filosofía modular, cualquier desarrollador podría añadir funcionalidades específicas sin modificar el código de la aplicación.

CAPÍTULO 3

OBJETIVOS

3.1. Objetivos principales

- Desarrollar un servidor web en Java desde cero.
- Permitir múltiples conexiones de manera simultánea, sin que el rendimiento se vea afectado.
- Soporte para protocolos *HTTP* y *HTTPS*.
- Permitir la extensión de funcionalidades mediante la implementación de módulos por parte del usuario sin modificar el código.

3.2. Objetivos secundarios

- Optimizar el rendimiento del servidor utilizando hilos virtuales gestionados exclusivamente por la *JVM* frente a hilos tradicionales, gestionados por el sistema operativo.

CAPÍTULO 4

RECURSOS

4.1. Software

Para el desarrollo del proyecto se han empleado distintas herramientas y recursos software. A continuación se describen las principales herramientas usadas:

- **Editor de código:** se ha utilizado *IntelliJ* como entorno de desarrollo principal al ser este el programa utilizado durante el periodo de formación en empresa.
- **Lenguaje de programación y librerías:**
 - *Java*: Lenguaje de programación principal para el desarrollo.
 - *Maven*: Herramienta utilizada para agilizar y facilitar la gestión de dependencias y la construcción del proyecto.
 - *Reflections*: Librería que permite escanear paquetes, usado para descubrir los módulos incluidos por el usuario.
 - *Lombok*: Librería que ayuda a reducir el código repetitivo mediante el uso de anotaciones para generar código automáticamente durante la compilación.
 - *Log4j2*: Librería para logging utilizada en el proyecto.
- **Control de versiones:**
 - *Git*: Herramienta utilizada para el control de versiones del código fuente, permitiendo llevar un seguimiento de los cambios realizados durante el desarrollo.
 - *GitHub*: Se utilizó para alojar el repositorio del proyecto, permitiendo así desarrollarlo y compartirlo entre distintos equipos.

4.2. Hardware

Para el desarrollo y documentación del proyecto se han utilizado diferentes dispositivos que han permitido trabajar en diversos entornos y situaciones. A continuación se describen los principales recursos hardware empleados:

- **Sobremesa personal:** ordenador personal de alto rendimiento utilizado como equipo principal para el desarrollo del proyecto y redacción de la memoria.
- **Portátil personal:** ordenador portátil personal utilizado de forma complementaria para continuar el desarrollo en movilidad o fuera del entorno de trabajo habitual.

Equipo	Procesador	Memoria RAM	Sistema operativo
Sobremesa personal	AMD Ryzen 7 2700X	32 GB DDR4	Windows 11 24H2
Portátil personal	Intel Core i5-1135G7	16 GB DDR4	Arch Linux

4.3. Humanos

El desarrollo de este proyecto ha sido llevado a cabo de manera individual, no obstante, durante el proceso se ha contado con la orientación y supervisión de Daniel Sierra Solís, quien ha proporcionado apoyo técnico durante el desarrollo.

CAPÍTULO 5

DESARROLLO DEL PROYECTO

5.1. Requisitos

Los requisitos se han definido a partir de los objetivos del proyecto y de las funcionalidades implementadas en el código. Se dividen en requisitos funcionales, requisitos no funcionales y restricciones de alcance.

a. Requisitos funcionales

Requisito	Descripción
Arranque del servidor	El sistema debe poder iniciarse desde la clase principal <i>JHTTP</i> .
Carga de configuración	El servidor debe cargar sus parámetros desde ficheros <i>.properties</i> . <i>application.properties</i> contiene valores generales y por defecto, mientras que <i>server.properties</i> contiene la configuración concreta de ejecución.
Validación del directorio raíz	El servidor debe validar el directorio configurado en <i>root.directory</i> . Dicho directorio debe existir, ser legible y ser un directorio válido. En caso contrario, el arranque debe detenerse con un error de configuración.
Configuración del puerto HTTP	El sistema debe permitir configurar el puerto HTTP mediante la propiedad <i>http.port</i> . Si no se indica un valor válido, debe utilizarse el puerto por defecto definido en <i>application.properties</i> .
Soporte opcional de HTTPS	El servidor debe permitir habilitar o deshabilitar HTTPS mediante la propiedad <i>https.disabled</i> . Cuando HTTPS está habilitado, debe crear un socket SSL en el puerto configurado mediante <i>https.port</i> .
Carga de certificado HTTPS	Cuando HTTPS está habilitado, el sistema debe cargar un certificado en formato PKCS12 desde la ruta indicada por <i>https.certificate.path</i> , utilizando la contraseña definida en <i>https.certificate.password</i> .
Redirección de HTTP a HTTPS	Si HTTPS está habilitado, las peticiones recibidas por el puerto HTTP no se procesan mediante el pipeline normal, sino que se redirigen al puerto HTTPS mediante una respuesta de redirección permanente.
Aceptación de múltiples conexiones	El servidor debe poder aceptar múltiples conexiones de clientes. Para ello, se mantiene un hilo encargado de aceptar conexiones y se delega cada conexión aceptada a un ejecutor.

Requisito	Descripción
Procesamiento concurrente	Cada conexión debe procesarse de forma independiente. El proyecto utiliza hilos virtuales para ejecutar las tareas asociadas a las conexiones, reduciendo el coste respecto a un modelo basado exclusivamente en hilos del sistema operativo.
Decodificación de peticiones HTTP	El sistema debe leer la petición HTTP desde el socket, detectar el final de las cabeceras mediante la secuencia <i>CRLF CRLF</i> , interpretar la línea inicial, extraer método, URI y versión, y almacenar las cabeceras recibidas.
Lectura del cuerpo de la petición	Cuando la petición contiene la cabecera <i>Content-Length</i> , el servidor debe leer del flujo de entrada el número de bytes indicado y almacenarlo como cuerpo de la petición.
Representación interna de peticiones	Las peticiones deben representarse mediante un objeto de dominio (<i>HttpRequest</i>) que agrupe método, URI, versión HTTP, cabeceras y cuerpo.
Representación interna de respuestas	Las respuestas deben representarse mediante un objeto de dominio (<i>HttpResponse</i>) que agrupe versión HTTP, código de estado, cabeceras y cuerpo.
Codificación de respuestas HTTP	El sistema debe transformar un <i>HttpResponse</i> en una respuesta HTTP válida, escribiendo línea de estado, cabeceras, línea en blanco y cuerpo en el flujo de salida del socket.
Cálculo de longitud de contenido	Antes de enviar la respuesta, el servidor debe establecer la cabecera <i>Content-Length</i> con el tamaño real del cuerpo de la respuesta.
Servicio de contenido estático	El servidor debe servir ficheros estáticos ubicados dentro del directorio raíz configurado. La URI solicitada se traduce a una ruta del sistema de ficheros y se lee el contenido si el fichero existe, es legible y no es un directorio. Ver figura 1.
Resolución de <i>index.html</i>	Cuando la ruta solicitada termina en /, el servidor debe intentar resolverla como una petición a <i>index.html</i> , permitiendo servir páginas de inicio de directorio.
Detección de tipo de contenido	El sistema debe asignar la cabecera <i>Content-Type</i> de acuerdo con la extensión del fichero solicitado. Se contemplan tipos frecuentes como HTML, CSS, JavaScript, JSON, imágenes, PDF, texto plano y otros recursos habituales.
Respuesta 404	Si ningún componente del pipeline genera una respuesta, el servidor debe devolver una página de error <i>404 Not Found</i> . Ver figura 2.

Requisito	Descripción
Respuesta 500	Si se produce una excepción no controlada durante el procesamiento de una conexión, el servidor debe intentar devolver una respuesta <i>500 Internal Server Error</i> y cerrar la conexión.
Plantilla de errores	Las páginas de error deben generarse a partir de una plantilla HTML reutilizable (<i>templates/error.html</i>) en la que se sustituyen código, mensaje, descripción y versión del servidor.
Pipeline de procesamiento	Las peticiones deben procesarse mediante una cadena de handlers. Cada handler recibe un contexto común con la petición, el socket y la posible respuesta.
Handlers internos	El servidor debe incluir handlers internos para contenido estático, redirección HTTPS y respuesta de fallback. Ver figura 3.
Sistema de módulos	El servidor debe permitir añadir funcionalidades mediante clases externas que implementen la interfaz <i>Handler</i> y estén anotadas con <i>@Module</i> .
Descubrimiento de módulos	Los módulos deben descubrirse en tiempo de arranque escaneando el paquete <i>sdevsantiago.jhttp.module.added</i> .
Activación de módulos por configuración	Los módulos disponibles deben activarse mediante la propiedad <i>modules</i> , que contiene una lista separada por comas con los nombres de los módulos que se desean insertar en el pipeline.
Registro de eventos	El servidor debe registrar eventos relevantes mediante Log4j2: arranque, puertos utilizados, conexiones aceptadas, respuestas enviadas, carga de módulos y errores.
Construcción ejecutable	El proyecto debe poder compilarse con Maven y generar un artefacto <i>.jar</i> ejecutable con la clase principal configurada.

b. Requisitos no funcionales

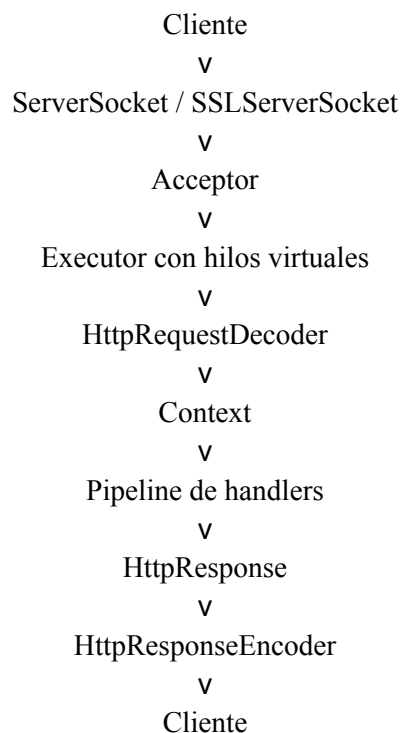
Categoría	Requisito	Descripción
Rendimiento	Concurrencia ligera	El sistema debe soportar múltiples conexiones simultáneas empleando hilos virtuales, una característica moderna de Java que permite un modelo de programación sencillo sin crear un hilo pesado del sistema operativo por cada cliente.

Categoría	Requisito	Descripción
Mantenibilidad	Separación de responsabilidades	La implementación debe separar configuración, arranque, transporte, decodificación HTTP, codificación HTTP, modelo de dominio, pipeline y handlers. Esta separación facilita comprender y modificar cada parte sin afectar al resto del sistema.
Extensibilidad	Módulos desacoplados	Nuevas funcionalidades deben poder añadirse mediante handlers externos, sin modificar el núcleo del servidor. El mecanismo de anotaciones permite registrar módulos con un nombre lógico y cargarlos desde configuración.
Configurabilidad	Parámetros externos	Los puertos, el directorio raíz, la activación de HTTPS, el certificado y los módulos no deben estar fijados en el código fuente, sino en ficheros de configuración.
Portabilidad	Ejecución sobre JVM	El servidor debe poder ejecutarse en cualquier sistema compatible con la versión de Java utilizada por el proyecto, siempre que disponga del directorio raíz y del certificado configurados cuando sea necesario.
Observabilidad	Logs claros	El sistema debe generar logs suficientes para saber si el servidor ha arrancado correctamente, qué puertos están activos, que conexiones se han aceptado y qué respuestas se han devuelto.
Robustez	Gestión de errores	El servidor debe controlar errores de configuración al arrancar y errores de procesamiento durante la atención de una conexión, evitando que un fallo puntual termine todo el proceso.
Seguridad	Comunicación cifrada opcional	El proyecto debe contemplar HTTPS mediante TLS, permitiendo cifrar la comunicación cuando se configura un certificado válido.
Seguridad	Reducción de exposición del sistema de ficheros	El servidor debe resolver las rutas solicitadas a partir de un directorio raíz concreto y normalizar las rutas antes de acceder al sistema de ficheros.

Categoría	Requisito	Descripción
Reproducibilidad	Construcción con Maven	La compilación debe estar definida en <i>pom.xml</i> , incluyendo versión de Java, dependencias, perfiles y configuración del artefacto ejecutable.

5.2. Arquitectura

La arquitectura de JHTTP se organiza en torno a una separación clara entre arranque, transporte, procesamiento HTTP, pipeline de handlers y modelo de dominio. Esta estructura permite que el servidor mantenga un núcleo relativamente pequeño, pero extensible mediante componentes independientes. A nivel general, el sistema sigue el siguiente flujo:



a. Arranque de la aplicación

El punto de entrada del servidor es la clase *JHTTP*. Esta clase carga las propiedades generales de la aplicación desde *application.properties*, donde se definen valores como la versión del proyecto, el puerto HTTP por defecto, el puerto HTTPS por defecto y el directorio raíz por defecto.

El método principal decide qué fichero de configuración utilizar. Si el usuario proporciona un argumento al ejecutar el programa, se carga ese fichero. Si no se proporciona argumento, se carga *server.properties*. La responsabilidad de construir la configuración se delega en *ConfigLoader*, que a su vez crea una instancia de *ServerConfig*.

ServerConfig centraliza la interpretación de las propiedades. Durante su construcción:

1. Carga el fichero de propiedades desde el classpath.
2. Obtiene el directorio raíz configurado.
3. Convierte dicho directorio en una ruta absoluta y normalizada.
4. Comprueba que el directorio existe, es legible y es realmente un directorio.
5. Lee el puerto HTTP.
6. Determina si HTTPS está habilitado.
7. Si HTTPS está habilitado, lee puerto HTTPS, ruta del certificado y contraseña.

Una vez creada la configuración, *JHTTP* instancia *ServerBootstrap* y llama a su método *start()*. A partir de este punto comienza el arranque real de la infraestructura de red.

b. Arquitectura HTTP y HTTPS

ServerBootstrap es la clase encargada de preparar el servidor. Su comportamiento depende de si HTTPS está habilitado o deshabilitado.

Cuando HTTPS está deshabilitado, el servidor:

1. Crea un *ServerSocket* asociado al puerto HTTP.
2. Construye el pipeline normal de procesamiento.
3. Crea un *Executor* asociado a ese pipeline.
4. Crea un *Acceptor* que escucha en el socket HTTP.
5. Arranca el acceptor en un hilo de plataforma.

Cuando HTTPS está habilitado, el servidor crea dos caminos diferenciados:

1. Un camino HTTPS real, asociado a un *SSLServerSocket*, donde se procesa la petición mediante el pipeline normal.
2. Un camino HTTP de redirección, asociado a un *ServerSocket* normal, donde las peticiones se transforman en respuestas de redirección hacia HTTPS.

Para crear el socket HTTPS, *ServerBootstrap* carga el certificado mediante *KeyStore*, inicializa un *KeyManagerFactory*, crea un *SSLContext* con protocolo TLS y obtiene de él una factoría capaz de generar el *SSLServerSocket*.

Esta decisión permite que el servidor acepte tráfico seguro cuando se configura un certificado y, al mismo tiempo, mantenga un puerto HTTP abierto para redirigir al usuario hacia la versión cifrada del mismo recurso.

c. Aceptación de conexiones y concurrencia

La aceptación de conexiones se realiza mediante la clase *Acceptor*. Esta clase implementa *Runnable* y mantiene un bucle que llama a *serverSocket.accept()*. Cada vez que un cliente se conecta:

1. Se acepta el socket del cliente.
2. Se registra en logs la dirección remota.
3. Se envía la conexión al *Executor*.

El *Executor* utiliza *Executors.newThreadPerTaskExecutor()* con una factoría de hilos virtuales. Esto significa que cada conexión se procesa en una tarea independiente, pero el coste de crear y mantener dichas tareas es menor que en un modelo clásico de hilo de sistema por conexión.

El modelo resultante es sencillo de razonar: el acceptor se dedica solo a aceptar conexiones, mientras que el executor se dedica a procesarlas. Esta separación evita que una petición lenta bloquee la aceptación de nuevas conexiones.

d. Ciclo de vida de una petición

El procesamiento de una conexión se concentra en *Executor.handleConnection()*. El flujo es el siguiente:

1. Se recibe el socket aceptado por el *Acceptor*.
2. *HttpRequestDecoder.readRequest()* lee la petición desde el flujo de entrada.
3. Se crea un *Context* asociado al socket.
4. Se almacena la petición en el contexto.
5. Se ejecuta el pipeline configurado.
6. Se obtiene la respuesta generada por el pipeline.
7. Se calcula y asigna la cabecera *Content-Length*.
8. *HttpResponseEncoder.sendResponse()* escribe la respuesta en el flujo de salida.
9. Se registran en logs la línea de petición, el estado de respuesta y la dirección del cliente.
10. Se cierra el socket en el bloque *finally*.

Este ciclo se repite de forma independiente para cada conexión aceptada. Si ocurre una excepción durante el proceso, el servidor registra el error e intenta responder con *500 Internal Server Error*.

e. Sistema de módulos

La arquitectura modular permite extender el comportamiento del servidor sin modificar su núcleo. Un módulo es una clase que cumple tres condiciones:

1. Implementa la interfaz *Handler*.
2. Está anotada con *@Module*.
3. Dispone de un constructor público sin argumentos.

Durante el arranque, *ModuleRegistry* escanea el paquete *sdevsantiago.jhttp.module.added* mediante la librería *Reflections*. Cada clase anotada con *@Module* y compatible con *Handler* se registra en un mapa interno, usando como clave el nombre declarado en la anotación.

Posteriormente, *PipelineModuleLoader* lee la propiedad *modules*, separa sus valores por comas, resuelve cada nombre contra el registro y devuelve una lista de handlers ya instalados. Estos handlers se insertan en el pipeline entre el handler de contenido estático y el fallback.

Esta arquitectura permite que el usuario añada comportamientos como autenticación, cabeceras personalizadas, respuestas dinámicas, registro adicional, restricciones por ruta o cualquier otra funcionalidad compatible con el modelo de petición y respuesta.

f. Modelo de datos HTTP

El servidor no trabaja directamente con cadenas de texto durante todo el procesamiento. En su lugar, transforma la información del protocolo en objetos propios:

- *HttpRequest*: representa método, URI, versión HTTP, cabeceras y cuerpo.
- *HttpResponse*: representa versión HTTP, estado, cabeceras y cuerpo.
- *HttpHeaders*: encapsula un mapa ordenado de cabeceras, permitiendo almacenar varios valores por nombre.
- *HttpMethod*: enumera métodos HTTP como *GET*, *POST*, *PUT*, *DELETE*, *HEAD*, *OPTIONS* y *PATCH*.
- *HttpVersion*: representa la versión HTTP soportada actualmente, *HTTP/1.1*.
- *HttpStatus*: enumera códigos de estado utilizados por el servidor, como *200 OK*, *308 Permanent Redirect*, *404 Not Found* y *500 Internal Server Error*.
- *ContentType*: asocia extensiones de fichero con tipos MIME.

Gracias a este modelo, el resto del sistema puede trabajar con conceptos HTTP de forma estructurada, evitando pasar cadenas sueltas entre componentes.

5.3. Construcción

Con el objetivo de agilizar y facilitar la construcción del proyecto, así como la inyección de dependencias, se utilizó *Maven*. Esta herramienta instala las dependencias que no se encuentren presentes en el equipo y las incluye en el archivo ejecutable. Para ello, se deberá utilizar el siguiente comando:

```
mvn clean package
```

En el archivo de configuración del proyecto (*pom.xml*) se incluyen dos perfiles por los que poder compilar el código, *dev* y *prod*, distinguiendo así los entornos de desarrollo y producción, respectivamente, empleados en multitud de proyectos. Para compilar el proyecto para producción se deberá añadir la siguiente instrucción al comando previamente indicado:

```
-P prod
```

5.4. Despliegue

Una vez compilado, el ejecutable se encontrará en el directorio */target*. Para ejecutarlo, será necesario introducir el siguiente comando:

```
java -jar target/jhttp-1.0.jar
```

Una vez hecho esto, el programa comenzará a ejecutarse como se ha descrito en este capítulo, pudiendo controlarse su funcionamiento en tiempo real mediante los logs en la terminal.

CAPÍTULO 6

CONCLUSIONES

El desarrollo de JHTTP ha permitido cumplir el objetivo principal planteado al inicio del proyecto: construir un servidor web funcional en Java partiendo de los elementos básicos proporcionados por el lenguaje y por el JDK, sin apoyarse en frameworks web ya existentes que oculten el funcionamiento interno del servidor. A lo largo del proyecto se ha trabajado directamente con sockets, flujos de entrada y salida, estructuras propias del protocolo HTTP, configuración externa, concurrencia, TLS y un sistema modular de procesamiento de peticiones.

El resultado obtenido es un servidor capaz de aceptar conexiones, interpretar peticiones, servir contenido estático desde un directorio configurado, generar respuestas HTTP válidas, devolver páginas de error, registrar eventos de ejecución, habilitar HTTPS mediante certificado y ampliar su comportamiento mediante módulos. Aunque no pretende sustituir a soluciones profesionales como Apache, NGINX o Tomcat, sí cumple su finalidad académica y técnica: demostrar que es posible implementar desde cero los mecanismos esenciales de un servidor web y comprender de forma práctica las decisiones que intervienen en su diseño.

6.1. Cumplimiento de objetivos

El primer objetivo principal consistió en desarrollar un servidor web en Java desde cero. Este objetivo se considera cumplido, ya que el proyecto implementa el ciclo completo de comunicación entre cliente y servidor. El servidor no delega el procesamiento HTTP en un framework externo, sino que incorpora sus propios componentes para decodificar peticiones, representar mensajes HTTP, ejecutar una cadena de handlers y codificar respuestas.

El segundo objetivo principal era permitir múltiples conexiones de manera simultánea sin que el rendimiento se viese afectado de forma significativa. Para ello se ha diseñado una separación entre aceptación de conexiones y procesamiento de peticiones. La clase *Acceptor* se encarga de aceptar sockets entrantes y el *Executor* procesa cada conexión mediante hilos virtuales. Esta solución permite mantener un modelo de programación sencillo, basado en operaciones bloqueantes, pero aprovechando una característica moderna de Java que reduce el coste asociado a crear una unidad de ejecución por conexión.

El tercer objetivo era proporcionar soporte para HTTP y HTTPS. El proyecto cumple este objetivo mediante dos caminos de ejecución. Cuando HTTPS está deshabilitado, el servidor atiende las peticiones directamente por el puerto HTTP. Cuando HTTPS está habilitado, se crea un *SSLServerSocket* a partir de un certificado y el puerto HTTP queda dedicado a redirigir las peticiones hacia el puerto HTTPS. De esta forma, el servidor puede funcionar tanto en modo simple como en modo cifrado.

El cuarto objetivo principal consistía en permitir la extensión de funcionalidades mediante módulos desarrollados por el usuario. Este objetivo se ha resuelto mediante una arquitectura basada en Handler, Pipeline, Context, la anotación `@Module`, *Module Registry* y

Pipeline ModuleLoader. Los módulos se descubren en tiempo de arranque, se registran por nombre y se activan mediante la propiedad *modules*. Gracias a esta decisión, el núcleo del servidor no necesita modificarse cada vez que se desea añadir una nueva funcionalidad.

En cuanto al objetivo secundario relacionado con el uso de hilos virtuales, también se considera alcanzado. El servidor utiliza `Thread.ofVirtual()` y `Executors.newThreadPerTaskExecutor()` para gestionar las conexiones. Esta decisión ha permitido profundizar en el modelo de concurrencia moderno de Java y compararlo conceptualmente con el uso tradicional de hilos de plataforma.

6.2. Futuras mejoras

Aunque el servidor cumple los objetivos establecidos, existen varias mejoras que podrían desarrollarse en versiones posteriores.

Otra mejora sería implementar respuestas específicas para peticiones mal formadas. Actualmente, muchos errores inesperados terminan en una respuesta genérica *500 Internal Server Error*. Sería conveniente detectar errores de parseo y responder con códigos más adecuados, como *400 Bad Request*, *405 Method Not Allowed* o *403 Forbidden*, según el caso.

También podría mejorarse el tratamiento de ficheros grandes. En la versión actual, el contenido del fichero se lee completo en memoria antes de enviarse al cliente. Para recursos de gran tamaño, sería más eficiente realizar streaming por bloques, reduciendo el consumo de memoria.

Otra línea de mejora sería implementar conexiones persistentes HTTP/1.1. El código ya contempla parcialmente la idea de comprobar la cabecera *Connection*, pero el ciclo actual cierra el socket al finalizar cada respuesta. Incorporar keep-alive permitirá atender varias peticiones sobre la misma conexión.

6.3. Conclusión final

Este proyecto ha sido especialmente útil para consolidar conocimientos de programación, redes, concurrencia, arquitectura software y seguridad básica en aplicaciones web. El desarrollo ha permitido pasar de una idea sencilla, crear un servidor HTTP en Java, a una implementación organizada que contempla configuración externa, HTTPS, pipeline de procesamiento y extensibilidad modular, siendo esta última la de mayor peso.

En definitiva, el proyecto cumple su propósito académico y técnico. Se ha obtenido un servidor funcional, ampliable y suficientemente estructurado, que demuestra la viabilidad de construir una solución HTTP/HTTPS desde sus fundamentos y deja abiertas líneas claras de evolución para futuras versiones.

REFERENCIAS

- MDN, *HTTP Messages*. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Messages>
- Oracle, *Virtual Threads*. Disponible en: <https://docs.oracle.com/en/java/javase/21/core/virtual-threads.html>
- ronmamo, *Reflections*. Disponible en: <https://github.com/ronmamo/reflections>
- baeldung, *A guide to the Reflections Library*. Disponible en: <https://www.baeldung.com/reflections-library>

FIGURAS

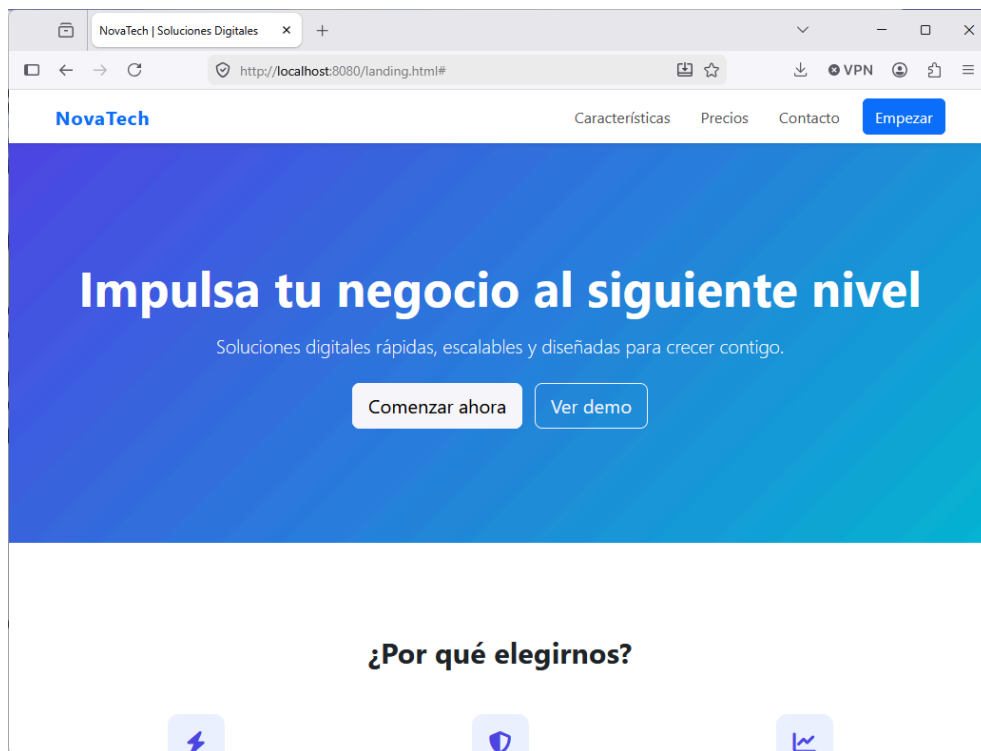


Figura 1. Ejemplo de contenido estático.

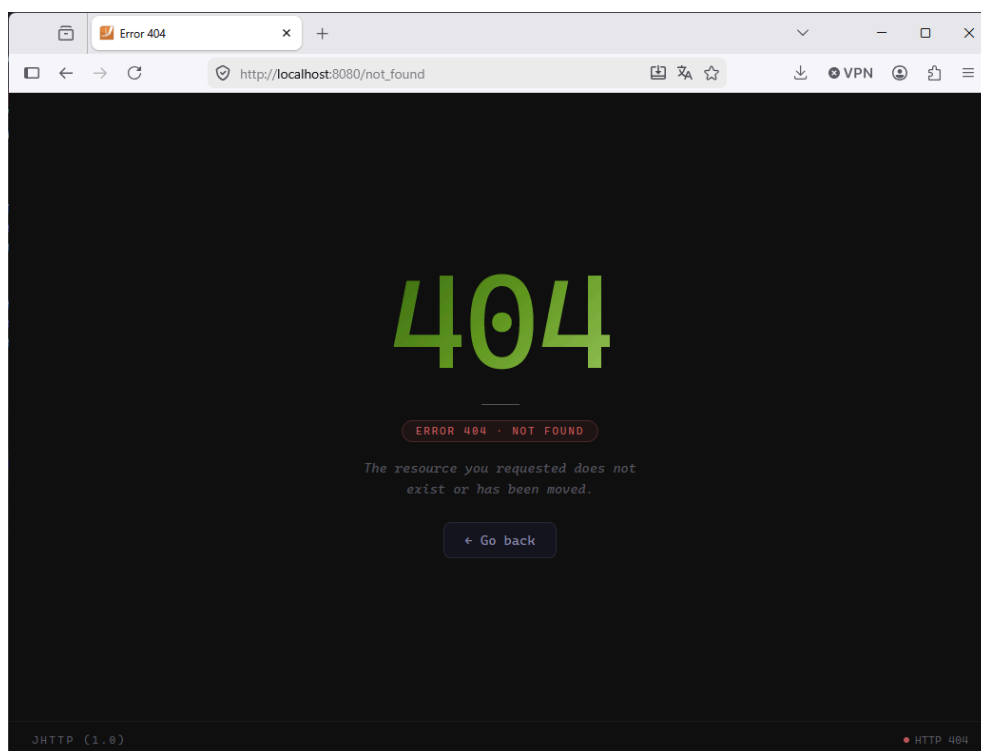


Figura 2. Error 404 generado por el FallbackHandler.

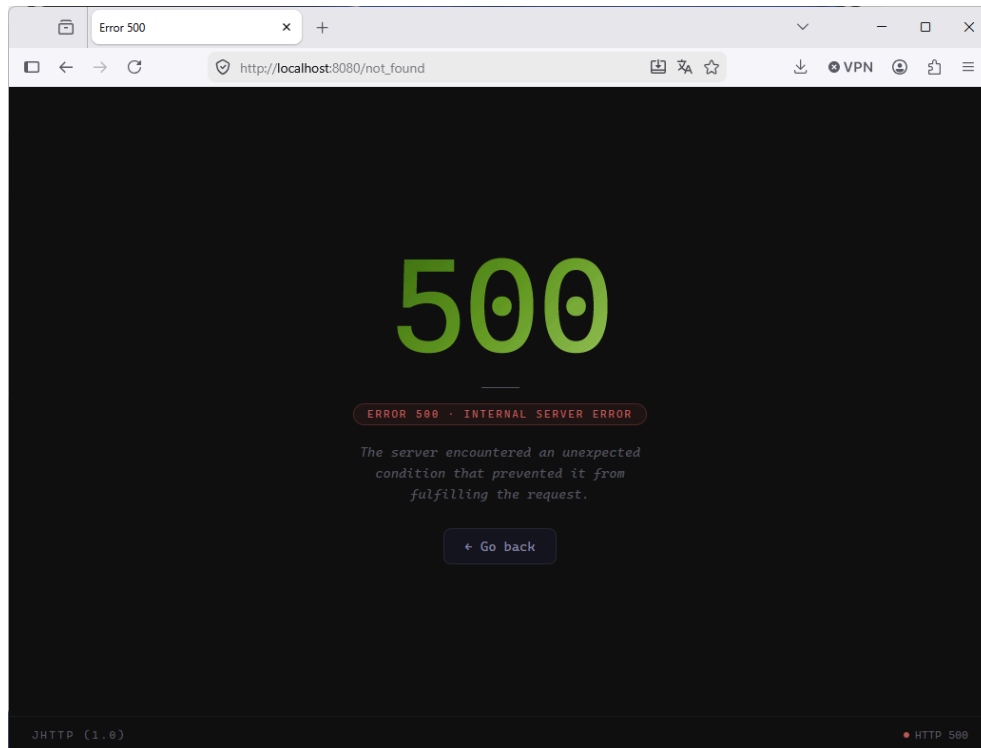


Figura 3. *Error 500.*